PROBLEM 10 · SOLUTION

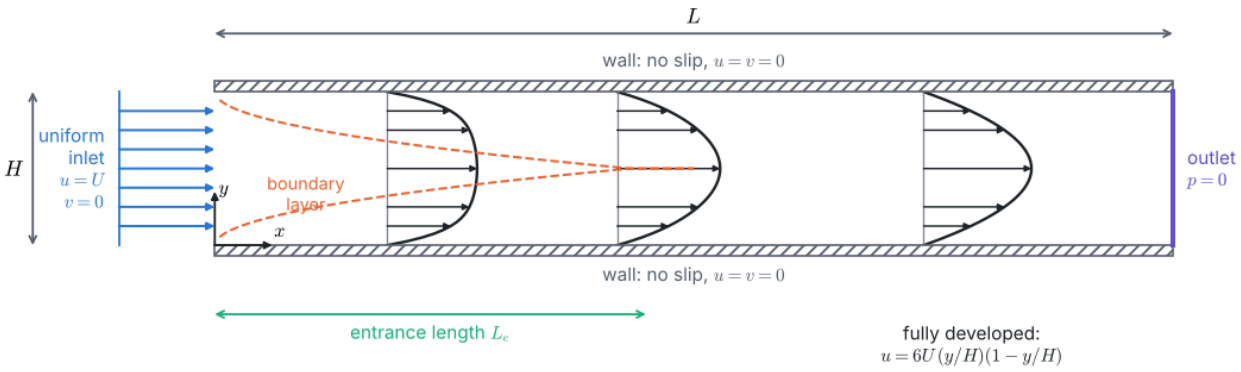
Developing laminar flow in a channel

Step-by-step solution with the Finite Difference Method

Plan. We follow the 7 pillars of CFD: geometry → grid → physics → numerical scheme → solution setup → compute → post-processing. All code below is plain Python (lists and loops only; `math` and `time` from Python itself). It is the code of the notebook `03_fdm_solver.ipynb`, where it can be run cell by cell.

Step 1 · Geometry and data

Two parallel plates a distance H apart; the fluid enters at $x = 0$ with a uniform velocity U and leaves at $x = L = 10\,H$ into a space at $p = 0$. Boundary layers grow on both plates, the core accelerates, and after the **entrance length** L_e the profile is the parabola of Poiseuille flow.



The channel and its boundary conditions.

In dimensionless form (lengths in H, velocities in U, time in H/U, pressure in ρU^2) only the **Reynolds number** $Re = \rho U H/\mu$ remains.

```
Re_values = [50, 100]          # the two Reynolds numbers of the problem
length = 10.0                  # channel length L in units of H

# a real example: water in a 1 mm gap, as in a compact plate heat exchanger or a microfluidic chip
rho, mu = 998.0, 1.0e-3        # density [kg/m^3] and viscosity [Pa s] of water at 20 °C
H_gap = 1.0e-3                 # gap between the plates [m]
for Re in Re_values:
    U = Re * mu / (rho * H_gap)
    print(f"Re = {Re:3d}: water in a {H_gap * 1000:.0f} mm gap, mean velocity U = {U * 100:.1f}
cm/s")
```

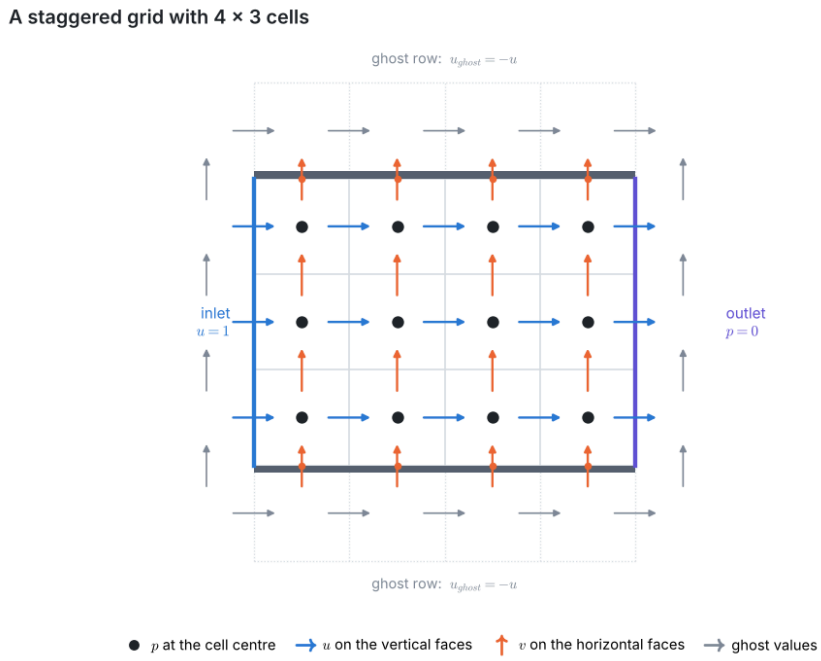
```
Re =  50: water in a 1 mm gap, mean velocity U = 5.0 cm/s
Re = 100: water in a 1 mm gap, mean velocity U = 10.0 cm/s
```

Step 2 · The grid: why staggered?

Square cells of side $h = H/16$: 160 along the channel, 16 across.

On a **collocated** grid (u, v, p at the same nodes) the pressure gradient at a node is $(p_E - p_W)/(2h)$, which skips the node itself. A pressure that alternates from node to node (a **checkerboard**) then has zero gradient

everywhere: the momentum equations cannot see it and nothing removes it. On the **staggered (MAC) grid** of Harlow and Welch (1965), p sits at the cell centres, u on the vertical faces and v on the horizontal faces. The pressure difference that drives the flow through a face uses the two cells on either side, $(p_E - p_P)/h$, and the mass leaving a cell is what crosses its four faces.



The staggered grid. One extra row and column of ghost values (grey) carries the boundary conditions.

In the code, `p[j][i]` is the pressure of cell (i, j), `u[j][i]` the velocity on its **right** face (`u[j][0]` is the inlet, `u[j][nx]` the outlet) and `v[j][i]` the velocity on its **top** face (`v[0][i]` and `v[ny][i]` lie on the walls).

```
ny = 16                         # cells across the channel
h = 1 / ny                      # cell size (in units of H)
nx = round(length / h)          # cells along the channel

print(f"h = {h} H:  {nx} x {ny} cells")
print(f"unknowns: p {nx * ny},  u {(nx - 1) * ny} (+ {ny} on the outlet),  v {nx * (ny - 1)}")
```

```
h = 0.0625 H:  160 x 16 cells
unknowns: p 2560,  u 2544 (+ 16 on the outlet),  v 2400
```

Step 3 · Physics

$$\nabla \cdot \mathbf{u} = 0, \qquad \frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\,\mathbf{u} = -\nabla p + \frac{1}{Re}\,\nabla^2 \mathbf{u}.$$

The stream function–vorticity method of Problems 6 and 7 removes the pressure. Here we **want** the pressure (the pressure drop), and the inlet and outlet would need extra conditions for ψ and ω; so we solve for the **primitive variables** u, v, p. The pressure has no equation of its own: it is whatever keeps the velocity divergence-free.

Boundary	Velocity	Pressure
inlet	$u = 1$, $v = 0$	free: $\partial p/\partial x = 0$
walls	$u = v = 0$	free: $\partial p/\partial y = 0$
outlet	free: $\partial u/\partial x = \partial v/\partial x = 0$	$p = 0$

Where the velocity is given, the pressure is free, and where the pressure is given, the velocity is free.

Fully developed flow. With $v = 0$ and $\partial/\partial x = 0$, the x-momentum equation is $dp/dx = (1/Re)\, d^2u/dy^2$ = constant. With $u(0) = u(1) = 0$ and $\int_0^1 u\, dy = 1$: $u = 6y(1-y)$, $u_{max} = 1.5$, $dp/dx = -12/Re$.

Entrance length. Durst et al. (2005) fitted $L_e/H = [0.631^{1.6} + (0.0442\, Re)^{1.6}]^{1/1.6}$ to their computations of channels with a uniform inlet (L_e: centre-line velocity at 99 % of its fully developed value).

```python
def poiseuille(y):
    """Exact fully developed velocity u / U at height y / H."""
    return 6 * y * (1 - y)


def durst(Re):
    """Entrance length L_e / H of a plane channel with a uniform inlet, Durst et al. (2005)."""
    return (0.631**1.6 + (0.0442 * Re) ** 1.6) ** (1 / 1.6)


for Re in Re_values:
    print(f"Re = {Re:3d}:  u_max = {poiseuille(0.5):.2f},  dp/dx = -12/Re = {-12 / Re:.3f},"
          f"  entrance length (Durst et al.) L_e = {durst(Re):.3f} H")
```

```
Re =  50:  u_max = 1.50,  dp/dx = -12/Re = -0.240,  entrance length (Durst et al.) L_e = 2.391 H
Re = 100:  u_max = 1.50,  dp/dx = -12/Re = -0.120,  entrance length (Durst et al.) L_e = 4.542 H
```

Step 4 · Numerical scheme: the projection method

Each time step has three parts (Chorin, 1968):

1. **Predictor:** march the momentum equations without the pressure:
   $\mathbf{u}^* = \mathbf{u}^n + \Delta t\,[\nabla^2\mathbf{u}^n/Re - (\mathbf{u}^n\cdot\nabla)\mathbf{u}^n]$ (FTCS, central differences).
2. **Pressure:** we want $\mathbf{u}^{n+1} = \mathbf{u}^* - \Delta t\,\nabla p$ to be divergence-free. Taking the divergence gives the **pressure Poisson equation** $\nabla^2 p = \nabla\cdot\mathbf{u}^*/\Delta t$.
3. **Correction:** $\mathbf{u}^{n+1} = \mathbf{u}^* - \Delta t\,\nabla p$.

Predictor for u on a vertical face ($\bar v$ is the average of the four v around the face):

$$u^*_{j,i} = u_{j,i} + \Delta t\left[\frac{u_{j,i+1}+u_{j,i-1}+u_{j+1,i}+u_{j-1,i}-4u_{j,i}}{Re\,h^2} - u_{j,i}\,\frac{u_{j,i+1}-u_{j,i-1}}{2h} - \bar v\,\frac{u_{j+1,i}-u_{j-1,i}}{2h}\right],$$

and the same for v with $\bar u$. **Ghost values** give the boundary conditions: $u_{ghost} = -u_{inside}$ beyond each wall (so that their average, the wall velocity, is zero), $v_{ghost} = -v_{inside}$ before the inlet ($v = 0$ there), $v_{ghost} = v_{inside}$ beyond the outlet. The outlet u^* is copied from its neighbour ($\partial u/\partial x = 0$).

```python
def new_fields(nx, ny):
    """Plug flow (u = 1, v = 0, p = 0) on the staggered grid, with one layer of ghost values."""
    u = [[1.0] * (nx + 1) for _ in range(ny + 2)]   # u[j][i] at x = i h,         y = (j - 1/2) h
    v = [[0.0] * (nx + 2) for _ in range(ny + 1)]   # v[j][i] at x = (i - 1/2) h, y = j h
    p = [[0.0] * (nx + 2) for _ in range(ny + 2)]   # p[j][i] at x = (i - 1/2) h, y = (j - 1/2) h
    return u, v, p


def boundary_conditions(u, v, nx, ny):
    """Ghost values: no slip on the walls, v = 0 at the inlet, dv/dx = 0 at the outlet."""
    for i in range(nx + 1):
        u[0][i] = -u[1][i]              # bottom wall: (ghost + first row) / 2 = 0
        u[ny + 1][i] = -u[ny][i]        # top wall
    for j in range(ny + 1):
        v[j][0] = -v[j][1]              # inlet: v = 0 halfway between ghost and first column
        v[j][nx + 1] = v[j][nx]         # outlet: dv/dx = 0
```

```python
def predictor(u, v, nx, ny, h, dt, Re):
    """Step 1: march the momentum equations one time step without the pressure (FTCS)."""
    us = [row[:] for row in u]
    vs = [row[:] for row in v]
    for j in range(1, ny + 1):                      # u on the inner vertical faces
        for i in range(1, nx):
            v_bar = (v[j][i] + v[j][i + 1] + v[j - 1][i] + v[j - 1][i + 1]) / 4
            convection = (u[j][i] * (u[j][i + 1] - u[j][i - 1])
                          + v_bar * (u[j + 1][i] - u[j - 1][i])) / (2 * h)
            diffusion = (u[j][i + 1] + u[j][i - 1] + u[j + 1][i] + u[j - 1][i]
                         - 4 * u[j][i]) / (Re * h * h)
            us[j][i] = u[j][i] + dt * (diffusion - convection)
        us[j][nx] = us[j][nx - 1]                   # outlet: du/dx = 0
    for j in range(1, ny):                          # v on the inner horizontal faces
        for i in range(1, nx + 1):
            u_bar = (u[j][i - 1] + u[j][i] + u[j + 1][i - 1] + u[j + 1][i]) / 4
            convection = (u_bar * (v[j][i + 1] - v[j][i - 1])
                          + v[j][i] * (v[j + 1][i] - v[j - 1][i])) / (2 * h)
            diffusion = (v[j][i + 1] + v[j][i - 1] + v[j + 1][i] + v[j - 1][i]
                         - 4 * v[j][i]) / (Re * h * h)
            vs[j][i] = v[j][i] + dt * (diffusion - convection)
    return us, vs
```

Pressure equation of cell P, multiplied by h^2:

$$a_E\,(p_E - p_P) + a_W\,(p_W - p_P) + a_N\,(p_N - p_P) + a_S\,(p_S - p_P) = b_P = \frac{h}{\Delta t}(u^*_E - u^*_W + v^*_N - v^*_S),$$

where $u^*_E, u^*_W, v^*_N, v^*_S$ are the velocities on the four faces of the cell. All coefficients are 1 except at the boundary: $a = 0$ towards the inlet and the walls (the velocity there is given and must not be corrected, so $\partial p/\partial n = 0$), and $a_E = 2, p_E = 0$ at the outlet (the face with $p = 0$ is only half a cell away).

Correction: $u_{j,i} = u^*_{j,i} - \Delta t\,(p_{j,i+1} - p_{j,i})/h$, at the outlet face $u = u^* + \Delta t\, p_{j,n_x}/(h/2)$, and $v_{j,i} = v^*_{j,i} - \Delta t\,(p_{j+1,i} - p_{j,i})/h$.

```python
def pressure_source(us, vs, nx, ny, h, dt):
    """Right-hand side b = h (flow out of the cell) / dt of the pressure equation."""
    b = [[0.0] * (nx + 2) for _ in range(ny + 2)]
    for j in range(1, ny + 1):
        for i in range(1, nx + 1):
            b[j][i] = h * (us[j][i] - us[j][i - 1] + vs[j][i] - vs[j - 1][i]) / dt
    return b


def correct(u, v, us, vs, p, nx, ny, h, dt):
    """Step 3: u = u* - dt grad p on every face. Returns the largest change of the velocity."""
    change = 0.0
    for j in range(1, ny + 1):
        for i in range(1, nx):
            new = us[j][i] - dt * (p[j][i + 1] - p[j][i]) / h
            change = max(change, abs(new - u[j][i]))
            u[j][i] = new
        u[j][nx] = us[j][nx] - dt * (0.0 - p[j][nx]) / (h / 2)     # outlet face, p = 0 there
    for j in range(1, ny):
        for i in range(1, nx + 1):
            new = vs[j][i] - dt * (p[j + 1][i] - p[j][i]) / h
            change = max(change, abs(new - v[j][i]))
            v[j][i] = new
    return change


def divergence(u, v, nx, ny, h):
    """du/dx + dv/dy of every cell: the mass that leaves the cell per unit time and area."""
    return [[(u[j][i] - u[j][i - 1] + v[j][i] - v[j - 1][i]) / h for i in range(1, nx + 1)]
            for j in range(1, ny + 1)]
```

Step 5 · Solution setup

Time step. The FTCS predictor is stable if $\Delta t \le Re\, h^2/4$ (diffusion in 2D), $\Delta t \le 2/(Re\, u_{max}^2)$ (central convection, Problem 5) and $u_{max}\Delta t/h \le 1$ (CFL). We use $u_{max} = 1.5$ and a factor 0.8:

```python
def time_step(Re, h, u_max=1.5):
    """Stable time step of the FTCS predictor, with a safety factor 0.8."""
    diffusion = Re * h * h / 4          # 2D diffusion: dt <= h^2 / (4 nu)
    convection = 2 / (Re * u_max**2)    # central convection: dt <= 2 nu / u^2
    cfl = h / u_max                     # CFL: u dt / h <= 1
    return 0.8 * min(diffusion, convection, cfl)


for Re in Re_values:
    print(f"Re = {Re:3d}:  diffusion {Re * h * h / 4:.4f},  convection {2 / (Re * 1.5**2):.4f},"
          f"  CFL {h / 1.5:.4f}   ->  dt = {time_step(Re, h):.5f}")
```

```
Re =  50:  diffusion 0.0488,  convection 0.0178,  CFL 0.0417   ->  dt = 0.01422
Re = 100:  diffusion 0.0977,  convection 0.0089,  CFL 0.0417   ->  dt = 0.00711
```

Pressure equation: SOR plus a block correction. We solve it with SOR ($\omega = 1.7$), starting from the previous pressure. On a channel ten times longer than high, plain SOR is very slow: an error that shifts the pressure of a whole cross-section must be passed column by column to the outlet. Before each SOR sweep we therefore add a constant c_i to every cell of column i, chosen so that each column's equations **added up** are satisfied. The wall terms cancel in the sum, and what remains is one tridiagonal system,

$$n_y\, a_W\,(c_{i-1} - c_i) + n_y\, a_E\,(c_{i+1} - c_i) = R_i = \sum_{\text{column } i} r_P, \qquad r_P = b_P - \sum a\,(p_{nb} - p_P),$$

solved with the **Thomas algorithm** of Problem 1. After the correction the divergence of a cell is $\Delta t\, r_P / h^2$, so we stop when all residuals are small enough for $|\nabla \cdot \mathbf{u}| < 10^{-6}$.

```
def thomas(a, b, c, r):
    """Solve a tridiagonal system directly (Thomas algorithm, Problem 1)."""
    n = len(b)
    c_star = [0.0] * n
    r_star = [0.0] * n
    c_star[0] = c[0] / b[0]
    r_star[0] = r[0] / b[0]
    for i in range(1, n):                       # forward sweep
        denominator = b[i] - a[i] * c_star[i - 1]
        c_star[i] = c[i] / denominator
        r_star[i] = (r[i] - a[i] * r_star[i - 1]) / denominator
    x = [0.0] * n
    x[n - 1] = r_star[n - 1]
    for i in range(n - 2, -1, -1):              # back substitution
        x[i] = r_star[i] - c_star[i] * x[i + 1]
    return x


def solve_pressure(p, b, nx, ny, limit, omega=1.7, block=True, max_sweeps=100_000):
    """Solve  a_E (p_E - p_P) + a_W (p_W - p_P) + a_N (p_N - p_P) + a_S (p_S - p_P) = b_P
    by SOR, with a block correction of every column before each sweep.
    Stops when every residual is below `limit`; returns the number of sweeps."""
    for sweep in range(max_sweeps + 1):
        # residual r_P of every cell, its largest size and its sum R_i over each column
        largest = 0.0
        R = [0.0] * nx
        for j in range(1, ny + 1):
            aN = 1.0 if j < ny else 0.0                 # top wall: dp/dy = 0
            aS = 1.0 if j > 1 else 0.0                  # bottom wall: dp/dy = 0
            for i in range(1, nx + 1):
                aW = 1.0 if i > 1 else 0.0              # inlet: dp/dx = 0
                aE, pE = (1.0, p[j][i + 1]) if i < nx else (2.0, 0.0)   # outlet: p = 0, h/2 away
                r = b[j][i] - (aE * (pE - p[j][i]) + aW * (p[j][i - 1] - p[j][i])
                               + aN * (p[j + 1][i] - p[j][i]) + aS * (p[j - 1][i] - p[j][i]))
                R[i - 1] += r
                largest = max(largest, abs(r))
        if largest < limit:                             # converged
            return sweep
        if not math.isfinite(largest):
            raise ArithmeticError("the pressure iteration diverged")

        if block:   # block correction: add c_i to column i (one tridiagonal system, Thomas)
            west = [0.0] + [float(ny)] * (nx - 1)       # n_y a_W: no west neighbour at the inlet
            east = [float(ny)] * (nx - 1) + [0.0]       # n_y a_E: p = 0 beyond the outlet
            centre = [-2.0 * ny] * nx                   # -n_y (a_W + a_E)
            centre[0] = -1.0 * ny                       # inlet column:  a_W + a_E = 0 + 1
            centre[-1] = -3.0 * ny                      # outlet column: a_W + a_E = 1 + 2
            c = thomas(west, centre, east, R)
            for j in range(1, ny + 1):
                for i in range(1, nx + 1):
                    p[j][i] += c[i - 1]

        for j in range(1, ny + 1):                      # one SOR sweep
            aN = 1.0 if j < ny else 0.0
            aS = 1.0 if j > 1 else 0.0
            for i in range(1, nx + 1):
                aW = 1.0 if i > 1 else 0.0
                aE, pE = (1.0, p[j][i + 1]) if i < nx else (2.0, 0.0)
                p_gs = ((aE * pE + aW * p[j][i - 1] + aN * p[j + 1][i] + aS * p[j - 1][i] - b[j][i])
                        / (aE + aW + aN + aS))
                p[j][i] += omega * (p_gs - p[j][i])
    return max_sweeps
```

The first pressure equation (plug flow suddenly meeting the walls) on the coarse grid $h = H/8$, with and without the block correction:

```
def first_pressure(Re, length, ny, block):
    """Sweeps needed for the pressure equation of the first time step."""
    h = 1 / ny
    nx = round(length / h)
    dt = time_step(Re, h)
    u, v, p = new_fields(nx, ny)
    boundary_conditions(u, v, nx, ny)
    us, vs = predictor(u, v, nx, ny, h, dt, Re)
    b = pressure_source(us, vs, nx, ny, h, dt)
    start = time.time()
    sweeps = solve_pressure(p, b, nx, ny, 1e-6 * h * h / dt, block=block)
    return sweeps, time.time() - start


for block in (False, True):
    sweeps, seconds = first_pressure(50, length, 8, block)
    print(f"block correction {'on ' if block else 'off'}: {sweeps:5d} SOR sweeps ({seconds:.2f} s)")
```

```
block correction off:  7476 SOR sweeps (3.83 s)
block correction on :    33 SOR sweeps (0.02 s)
```

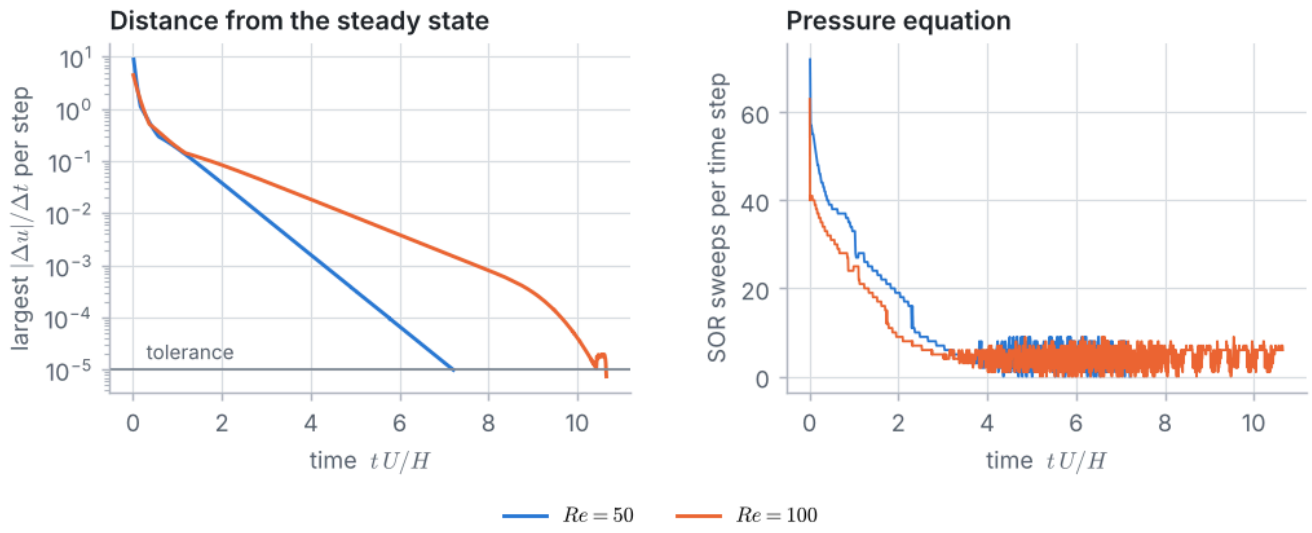
The solver. Start from plug flow ($u = 1$, $v = 0$, $p = 0$) and march until the largest $|\Delta u|/\Delta t$ in a step is below 10^{-5}: then $\partial u/\partial t = 0$, and u, v, p satisfy the steady equations.

```
def solve_channel(Re, length, ny, tol=1e-5, max_steps=100_000):
    """Developing channel flow: march with the projection method until the flow is steady."""
    h = 1 / ny
    nx = round(length / h)
    dt = time_step(Re, h)
    u, v, p = new_fields(nx, ny)
    history = []                    # (largest |du/dt|, pressure sweeps) of every step
    for step in range(max_steps):
        boundary_conditions(u, v, nx, ny)
        us, vs = predictor(u, v, nx, ny, h, dt, Re)                        # 1 · predictor
        b = pressure_source(us, vs, nx, ny, h, dt)
        sweeps = solve_pressure(p, b, nx, ny, 1e-6 * h * h / dt)           # 2 · pressure
        change = correct(u, v, us, vs, p, nx, ny, h, dt)                    # 3 · correction
        if not math.isfinite(change):
            raise ArithmeticError(f"the time march diverged after {step + 1} steps")
        history.append((change / dt, sweeps))
        if change / dt < tol:       # steady state
            break
    boundary_conditions(u, v, nx, ny)
    return u, v, p, history, dt
```

Step 6 · Compute

```
solutions = {}
for Re in Re_values:
    start = time.time()
    u, v, p, history, dt = solve_channel(Re, length, ny)
    solutions[Re] = (u, v, p, history, dt)
    print(f"Re = {Re:3d}: steady after {len(history)} time steps (t = {len(history) * dt:.1f} H/U),"
          f" {sum(s for _, s in history)} pressure sweeps  ({time.time() - start:.1f} s)")
```

```
Re =  50: steady after 507 time steps (t = 7.2 H/U), 6765 pressure sweeps  (16.1 s)
Re = 100: steady after 1498 time steps (t = 10.7 H/U), 12788 pressure sweeps  (33.7 s)
```



Left: distance from the steady state. Right: SOR sweeps per time step; the previous pressure is an excellent first guess once the flow has settled.

The start-up disturbance is carried out through the outlet within about one flow-through time ($t \approx L/U$). At $Re = 100$ it is damped less on the way and its last part leaves at $t \approx 10$, where the curve drops sharply.

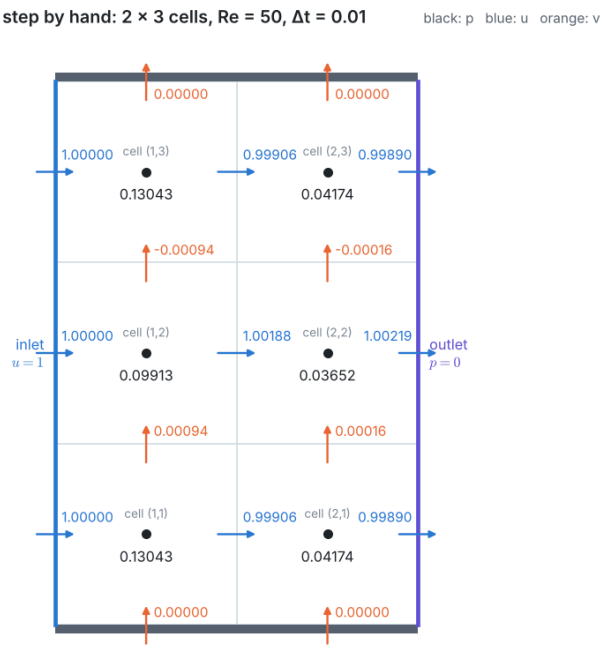
The whiteboard numbers. The whiteboard guide does one projection step by hand on 2 × 3 cells ($h = 1/3$, $Re = 50$, $\Delta t = 0.01$). The same step with the notebook's functions:

```
def show(name, field, rows, columns):
    """Print field[j][i] for the given rows (from the top down) and columns."""
    print(f"  {name}:")
    for j in reversed(rows):
        print(f"    j = {j}: " + "  ".join(f"{field[j][i] + 0.0:9.6f}" for i in columns))


Re_hand, nx_hand, ny_hand, dt_hand = 50, 2, 3, 0.01
h_hand = 1 / ny_hand
u_h, v_h, p_h = new_fields(nx_hand, ny_hand)
boundary_conditions(u_h, v_h, nx_hand, ny_hand)
us_h, vs_h = predictor(u_h, v_h, nx_hand, ny_hand, h_hand, dt_hand, Re_hand)
b_h = pressure_source(us_h, vs_h, nx_hand, ny_hand, h_hand, dt_hand)
sweeps_h = solve_pressure(p_h, b_h, nx_hand, ny_hand, 1e-13)
correct(u_h, v_h, us_h, vs_h, p_h, nx_hand, ny_hand, h_hand, dt_hand)

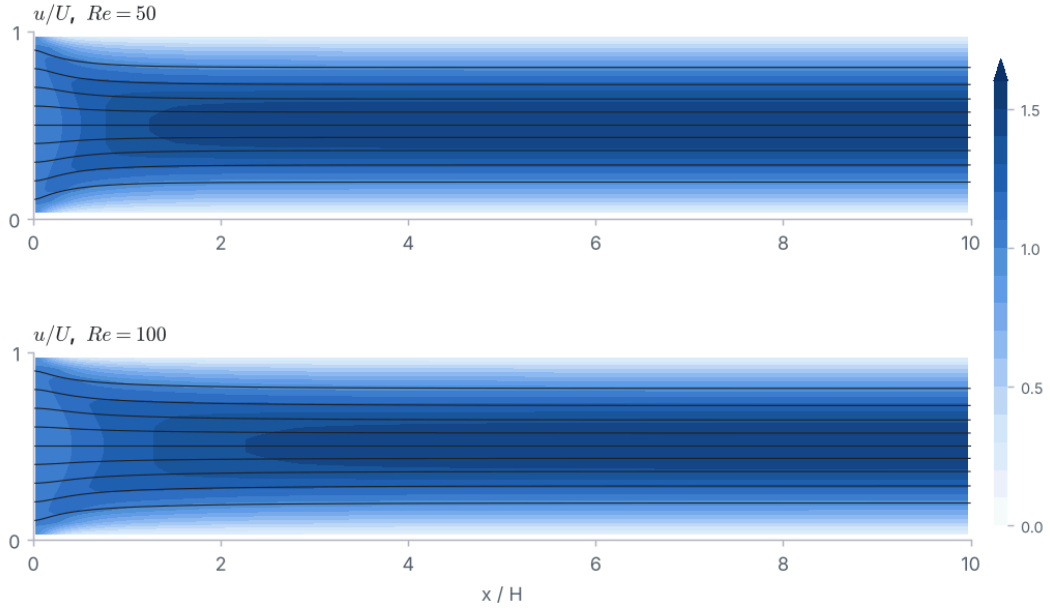
print("1 · predictor")
show("u* on the vertical faces (i = 0 inlet ... 2 outlet)", us_h, range(1, 4), range(0, 3))
print("2 · pressure equation")
show("b = h (flow out of the cell) / dt", b_h, range(1, 4), range(1, 3))
print(f"  pressure (SOR, {sweeps_h} sweeps):")
show("p", p_h, range(1, 4), range(1, 3))
print("3 · correction")
show("u", u_h, range(1, 4), range(0, 3))
show("v on the horizontal faces (j = 0 and 3 are the walls)", v_h, range(0, 4), range(1, 3))
div_h = divergence(u_h, v_h, nx_hand, ny_hand, h_hand)
print(f"largest |div u| after the step: {max(abs(d) for row in div_h for d in row):.1e}")
print(f"flow rate through the outlet: {h_hand * sum(u_h[j][nx_hand] for j in range(1, 4)):.6f}")
```

```
1 · predictor
  u* on the vertical faces (i = 0 inlet ... 2 outlet):
    j = 3:  1.000000   0.996400   0.996400
    j = 2:  1.000000   1.000000   1.000000
    j = 1:  1.000000   0.996400   0.996400
2 · pressure equation
  b = h (flow out of the cell) / dt:
    j = 3: -0.120000   0.000000
    j = 2:  0.000000   0.000000
    j = 1: -0.120000   0.000000
  pressure (SOR, 68 sweeps):
  p:
    j = 3:  0.130435   0.041739
    j = 2:  0.099130   0.036522
    j = 1:  0.130435   0.041739
3 · correction
  u:
    j = 3:  1.000000   0.999061   0.998904
    j = 2:  1.000000   1.001878   1.002191
    j = 1:  1.000000   0.999061   0.998904
  v on the horizontal faces (j = 0 and 3 are the walls):
    j = 3:  0.000000   0.000000
    j = 2: -0.000939  -0.000157
    j = 1:  0.000939   0.000157
    j = 0:  0.000000   0.000000
largest |div u| after the step: 7.0e-15
flow rate through the outlet: 1.000000
```



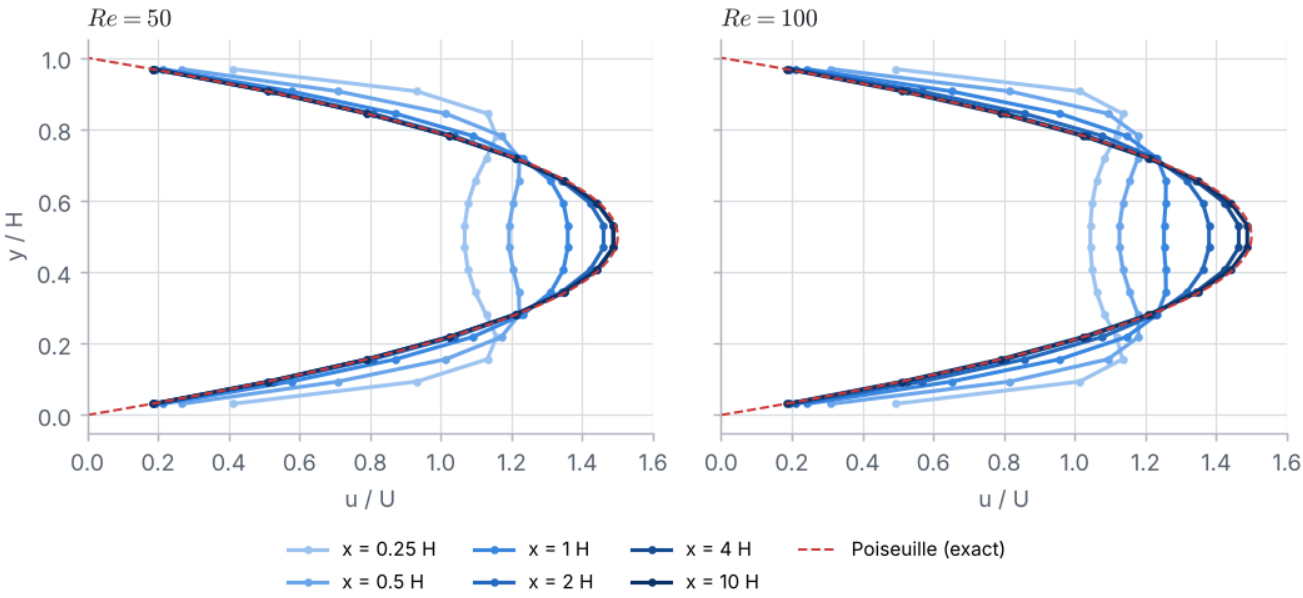
The hand grid after one projection step.

Step 7 · Post-processing and verification



black lines: streamlines, 10 % of the flow between two neighbouring lines (y stretched 2×)

The velocity $\boldsymbol{u}$ and streamlines (10 % of the flow between neighbouring lines). The fluid slows down near the walls and accelerates in the core; near the inlet the streamlines bend towards the centre.



Velocity profiles at several $\boldsymbol{x}$, approaching the Poiseuille parabola. Near the inlet the profile is flat in the middle with two small "ears" next to the walls.

Outlet profile. The fully developed flow **on our grid** solves $(u_{j+1} - 2u_j + u_{j-1})/h^2 = Re\, dp/dx$ with the wall ghosts and a flow rate of 1, a tridiagonal system (Thomas again):

```python
def developed_profile(ny, Re):
    """Fully developed flow on the grid: the discrete Poiseuille profile and its dp/dx."""
    h = 1 / ny
    below, diagonal, above = [1.0] * ny, [-2.0] * ny, [1.0] * ny
    below[0], above[-1] = 0.0, 0.0
    diagonal[0] = diagonal[-1] = -3.0              # ghost value u = -u beyond each wall
    shape = thomas(below, diagonal, above, [-1.0] * ny)   # u_{j+1} - 2 u_j + u_{j-1} = -1
    rate = h * sum(shape)
    profile = [s / rate for s in shape]            # scaled to a flow rate of 1
    dpdx = -1 / (Re * h * h * rate)
    return profile, dpdx


for Re in Re_values:
    profile, dpdx_grid = developed_profile(ny, Re)
    outlet = [solutions[Re][0][j][nx] for j in range(1, ny + 1)]
    error_exact = max(abs(outlet[k] - poiseuille(yc[k])) for k in range(ny))
    error_grid = max(abs(outlet[k] - profile[k]) for k in range(ny))
    print(f"Re = {Re:3d}: outlet profile minus exact Poiseuille: up to {error_exact:.4f};"
          f"  minus the grid's fully developed profile: up to {error_grid:.1e}")
profile, dpdx_grid = developed_profile(ny, 50)
centre = (profile[ny // 2 - 1] + profile[ny // 2]) / 2
print(f"fully developed flow on this grid: centre velocity {centre:.5f} (exact 1.5),"
      f"  Re dp/dx = {50 * dpdx_grid:.4f} (exact -12)")
```

```
Re =  50: outlet profile minus exact Poiseuille: up to 0.0058;  minus the grid's fully developed
profile: up to 6.0e-06
Re = 100: outlet profile minus exact Poiseuille: up to 0.0061;  minus the grid's fully developed
profile: up to 3.2e-04
fully developed flow on this grid: centre velocity 1.48837 (exact 1.5),  Re dp/dx = -11.9070 (exact
-12)
```

The outlet agrees with the grid's own fully developed flow, so the outlet is far enough downstream; the 0.4 % difference from the exact parabola is the discretization error of the grid, mostly from the ghost-value wall condition.

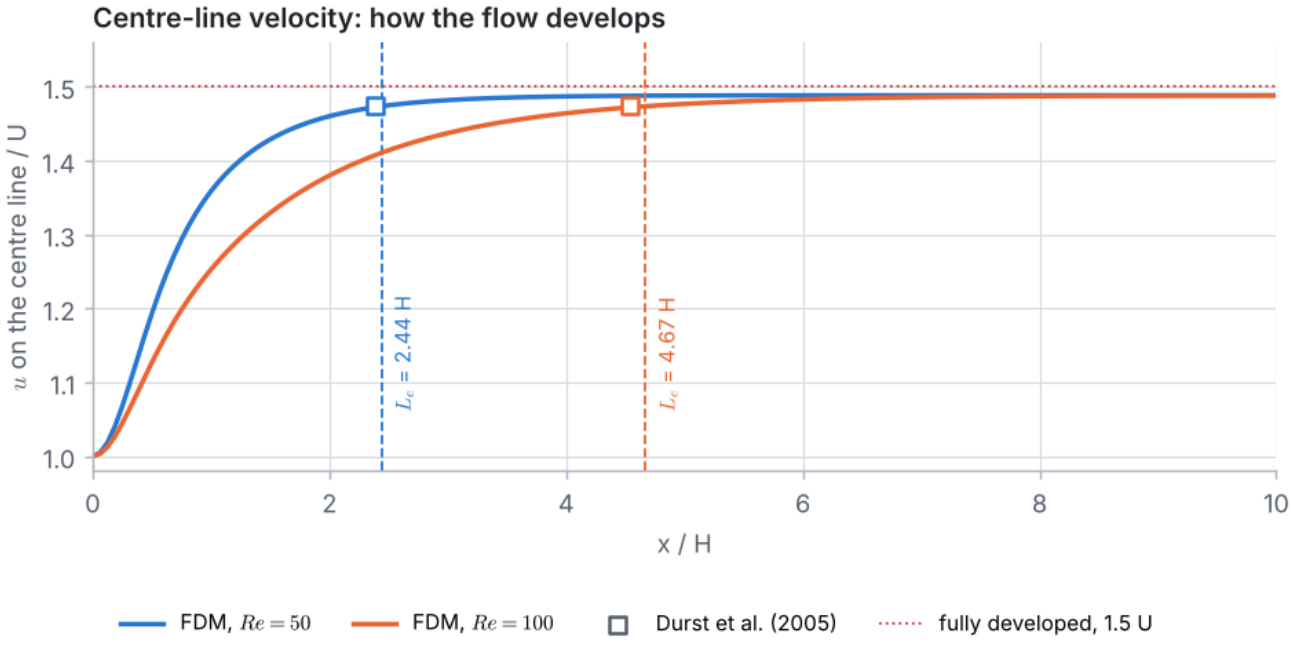
Entrance length. L_e is where the centre-line velocity reaches 99 % of the centre velocity of the grid's fully developed flow (so that the grid's error in the parabola itself does not count, and the outlet position does not matter):

```python
def centre_line(u, nx, ny):
    """u on the centre line y = 1/2 at every vertical face x = i h."""
    return [(u[ny // 2][i] + u[ny // 2 + 1][i]) / 2 for i in range(nx + 1)]


def entrance_length(line, h, developed):
    """Distance at which the centre-line velocity reaches 99 % of its fully developed value."""
    target = 0.99 * developed
    for i in range(1, len(line)):
        if line[i] >= target:
            return (i - 1 + (target - line[i - 1]) / (line[i] - line[i - 1])) * h


lines, L_e, developed = {}, {}, {}
for Re in Re_values:
    lines[Re] = centre_line(solutions[Re][0], nx, ny)
    profile, _ = developed_profile(ny, Re)
    developed[Re] = (profile[ny // 2 - 1] + profile[ny // 2]) / 2     # grid's Poiseuille centre
    L_e[Re] = entrance_length(lines[Re], h, developed[Re])
    difference = 100 * (L_e[Re] / durst(Re) - 1)
    print(f"Re = {Re:3d}: centre-line u at the outlet {lines[Re][-1]:.4f} (exact 1.5);"
          f"  L_e = {L_e[Re]:.3f} H,  Durst et al. {durst(Re):.3f} H  ({difference:+.1f} %)")
```

```
Re =  50: centre-line u at the outlet 1.4884 (exact 1.5);  L_e = 2.438 H,  Durst et al. 2.391 H
(+2.0 %)
Re = 100: centre-line u at the outlet 1.4880 (exact 1.5);  L_e = 4.669 H,  Durst et al. 4.542 H
(+2.8 %)
```



Centre-line velocity along the channel, with the computed entrance lengths (dashed) and the correlation of Durst et al. (squares).

Mass conservation. The flow rate $Q = \sum_j u_{j,i}\, h$ at several cross-sections, and the largest divergence of any cell:

```python
for Re in Re_values:
    u = solutions[Re][0]
    rates = [(x, h * sum(u[j][round(x / h)] for j in range(1, ny + 1)))
             for x in (0, 1, 2, 5, length)]
    div = divergence(u, solutions[Re][1], nx, ny, h)
    print(f"Re = {Re:3d}: flow rate " + ",  ".join(f"Q(x = {x:g}) = {q:.8f}" for x, q in rates))
    print(f"          largest |div u| in any cell: {max(abs(d) for row in div for d in row):.1e}")
```

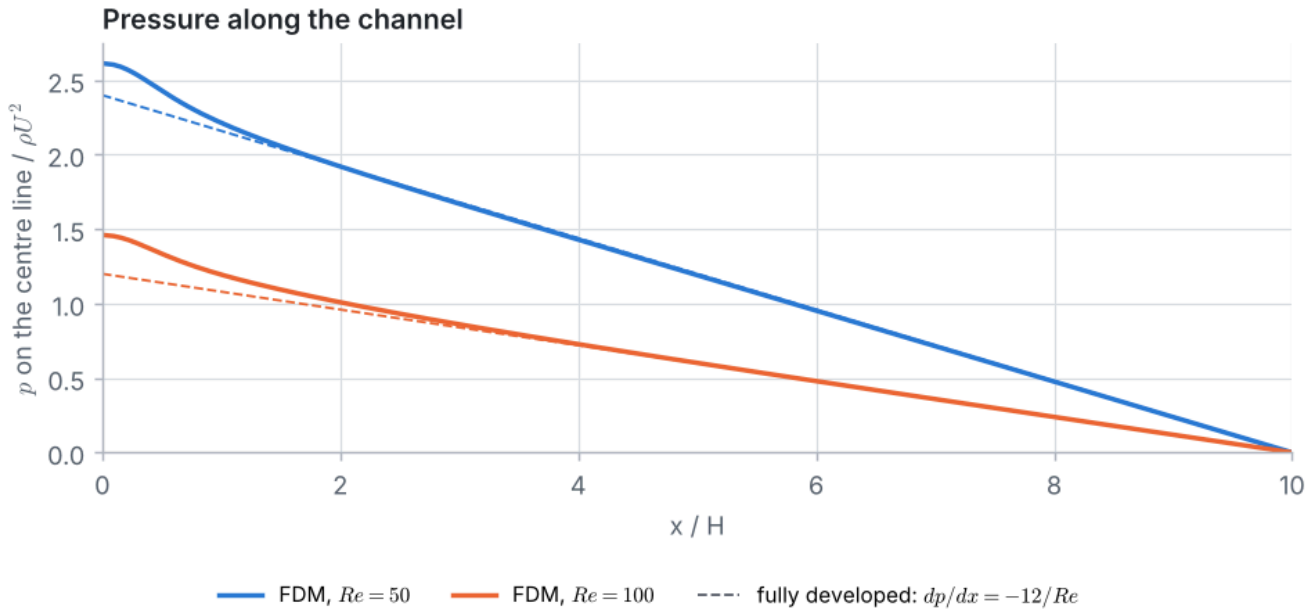
```
Re =  50: flow rate Q(x = 0) = 1.00000000,  Q(x = 1) = 1.00000000,  Q(x = 2) = 1.00000000,  Q(x = 5)
= 1.00000000,  Q(x = 10) = 1.00000000
          largest |div u| in any cell: 9.1e-07
Re = 100: flow rate Q(x = 0) = 1.00000000,  Q(x = 1) = 1.00000000,  Q(x = 2) = 1.00000000,  Q(x = 5)
= 1.00000000,  Q(x = 10) = 1.00000000
          largest |div u| in any cell: 9.2e-07
```

Pressure drop. We extrapolate the centre-line pressure to $x = 0$ and split it into the drop of fully developed flow over the whole length, $|dp/dx|\, L$ (gradient computed at the outlet), and the **extra drop** of the entrance region, $K\, \frac{1}{2}\rho U^2$:

```python
def centre_pressure(p, nx, ny):
    """p on the centre line at every cell centre x = (i - 1/2) h."""
    return [(p[ny // 2][i] + p[ny // 2 + 1][i]) / 2 for i in range(1, nx + 1)]


pressures, drops = {}, {}
for Re in Re_values:
    pc = centre_pressure(solutions[Re][2], nx, ny)
    p_inlet = 1.5 * pc[0] - 0.5 * pc[1]             # linear extrapolation to x = 0
    slope = (pc[-1] - pc[-2]) / h                    # dp/dx at the outlet
    pressures[Re] = [p_inlet] + pc + [0.0]
    drops[Re] = (p_inlet, slope)
    extra = p_inlet + slope * length
    print(f"Re = {Re:3d}: dp/dx at the outlet {slope:.4f} (exact {-12 / Re:.4f})")
    print(f"          inlet pressure {p_inlet:.4f} = {-slope * length:.4f} (fully developed)"
          f" + {extra:.4f} (extra),  K = {2 * extra:.3f}")
```

```
Re =  50: dp/dx at the outlet -0.2381 (exact -0.2400)
          inlet pressure 2.6139 = 2.3814 (fully developed) + 0.2325 (extra),  K = 0.465
Re = 100: dp/dx at the outlet -0.1191 (exact -0.1200)
          inlet pressure 1.4613 = 1.1906 (fully developed) + 0.2707 (extra),  K = 0.541
```



Centre-line pressure. Downstream it follows the straight line $dp/dx = -12/Re$ (dashed); near the inlet it falls faster: the core is accelerated and the thin boundary layers cause high wall friction.

The checkerboard, with numbers. We add a checkerboard of ± 0.1 to the converged pressure and compare the pressure gradients the momentum equations would see:

```
amplitude = 0.1
p50 = solutions[50][2]
p_check = [[p50[j][i] + amplitude * (-1) ** (i + j) for i in range(nx + 2)] for j in range(ny + 2)]



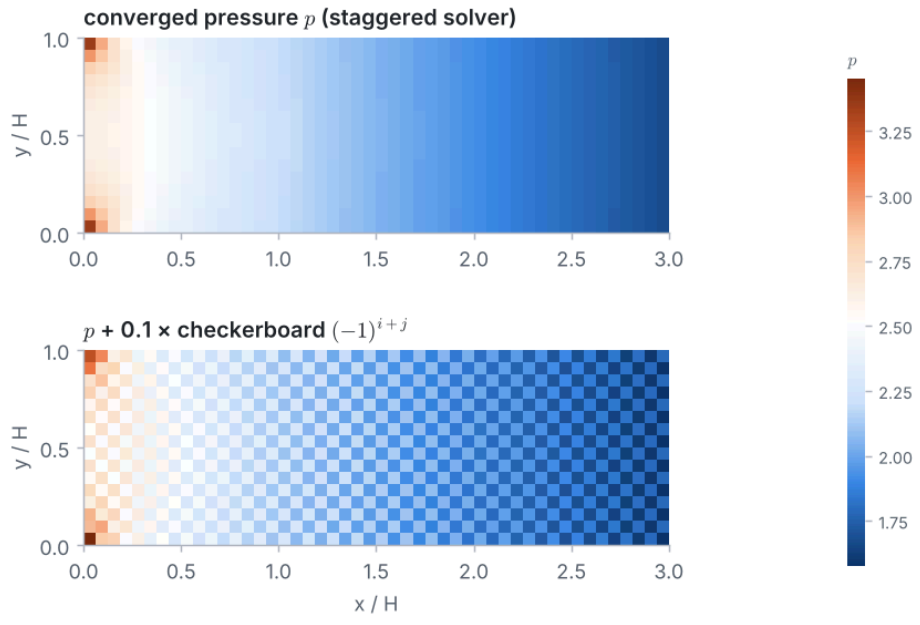
def gradient_collocated(q, i, j):
    """Pressure gradient at the centre of cell (i, j) on a collocated grid."""
    return (q[j][i + 1] - q[j][i - 1]) / (2 * h), (q[j + 1][i] - q[j - 1][i]) / (2 * h)


def gradient_staggered(q, i, j):
    """Pressure gradient on the right and top faces of cell (i, j) on the staggered grid."""
    return (q[j][i + 1] - q[j][i]) / h, (q[j + 1][i] - q[j][i]) / h


largest = {}
for name, gradient in (("collocated", gradient_collocated), ("staggered", gradient_staggered)):
    largest[name] = max(abs(a - b) for j in range(2, ny) for i in range(2, nx)
                        for a, b in zip(gradient(p50, i, j), gradient(p_check, i, j)))

print(f"largest change of the pressure gradient caused by the checkerboard:")
print(f"  collocated grid, (p_E - p_W) / 2h:  {largest['collocated']:.1e}")
print(f"  staggered grid,  (p_E - p_P) / h:   {largest['staggered']:.2f}")
```

```
largest change of the pressure gradient caused by the checkerboard:
  collocated grid, (p_E - p_W) / 2h:  1.8e-15
  staggered grid,  (p_E - p_P) / h:   3.20
```



On a collocated grid both fields give exactly the same pressure gradient, $(p_E - p_W)/(2h)$, at every node.

The converged pressure and the same pressure with a checkerboard added. A collocated solver cannot tell them apart. (Note the high pressure in the inlet corners, where the uniform inlet meets the walls: the pressure is singular there.)

On the collocated grid the checkerboard changes nothing (round-off); on the staggered grid it changes the gradient by $2 \times 0.1/h = 3.2$ and would be pushed away at once.

Grid study for $Re = 50$, on a channel $5\,H$ long (twice L_e; the check in the last line shows that the length does not matter):

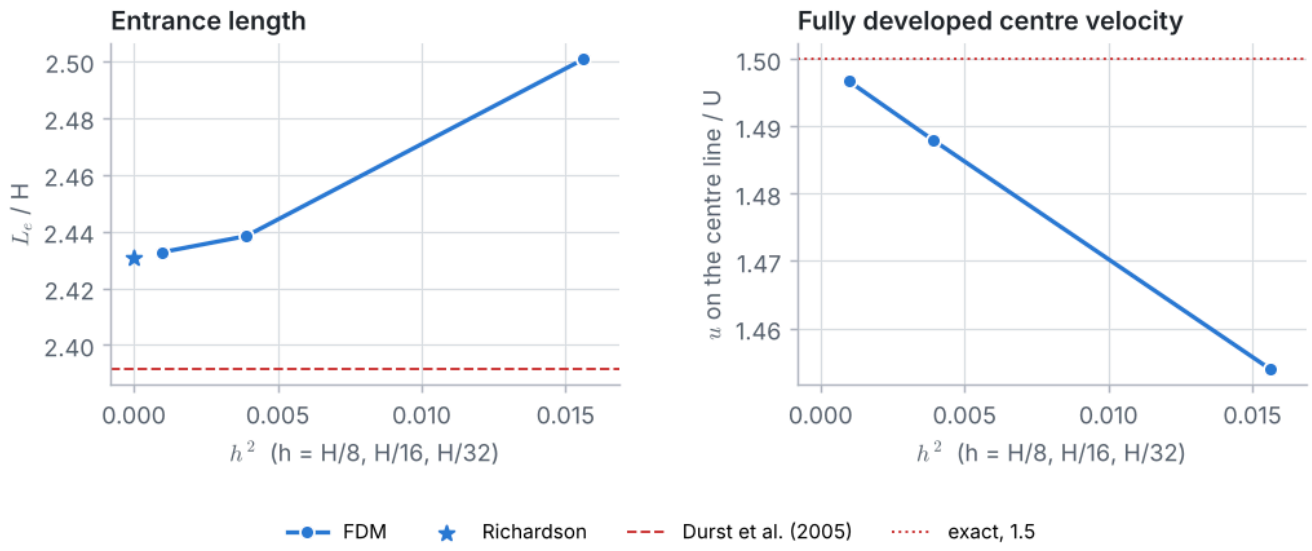
```
grids = [8, 16, 32]
short = 5.0
study = []
print("   h       steps  time [s]   L_e [H]   u_centre(outlet)   dp/dx (outlet)   extra drop")
for n in grids:
    start = time.time()
    u_n, v_n, p_n, hist_n, dt_n = solve_channel(50, short, n)
    nx_n = round(short * n)
    line = centre_line(u_n, nx_n, n)
    pc = centre_pressure(p_n, nx_n, n)
    profile, _ = developed_profile(n, 50)
    dpdx = (pc[-1] - pc[-2]) * n
    p_in = 1.5 * pc[0] - 0.5 * pc[1]
    result = {"Le": entrance_length(line, 1 / n, (profile[n // 2 - 1] + profile[n // 2]) / 2),
              "uc": line[-1], "dpdx": dpdx, "extra": p_in + dpdx * short}
    study.append(result)
    print(f"  1/{n:<3d} {len(hist_n):7d}  {time.time() - start:7.1f}   {result['Le']:7.4f}"
          f"       {result['uc']:.5f}          {result['dpdx']:.5f}       {result['extra']:.4f}")
print(f"  exact                                        1.50000          {-12 / 50:.5f}")
change = study[1]["Le"] - L_e[50]
print(f"  L = {length:g} H, h = 1/16:   L_e = {L_e[50]:.4f}")
print(f"  so the shorter channel changes L_e by {change:+.4f}")
```

```
   h       steps  time [s]   L_e [H]   u_centre(outlet)   dp/dx (outlet)   extra drop
  1/8       385      1.2    2.5008       1.45393          -0.23283       0.2497
  1/16      361      7.1    2.4384       1.48798          -0.23825       0.2318
  1/32      533     72.8    2.4327       1.49674          -0.23972       0.2231
  exact                                      1.50000          -0.24000
  L = 10 H, h = 1/16:   L_e = 2.4384
  so the shorter channel changes L_e by +0.0000
```

Richardson extrapolation, $f \approx f_{h/2} + (f_{h/2} - f_h)/3$, and the observed order $p = \log_2[(f_8 - f_{16})/(f_{16} - f_{32})]$:

```
print("                      h = 1/32   extrapolated   exact / correlation   observed order")
quantities = (("uc", "u_centre (outlet)", 1.5), ("dpdx", "dp/dx (outlet)", -12 / 50),
              ("Le", "entrance length", durst(50)), ("extra", "extra drop", None))
for name, label, reference in quantities:
    f8, f16, f32 = (s[name] for s in study)
    extrapolated = f32 + (f32 - f16) / 3
    order = math.log2((f8 - f16) / (f16 - f32))
    reference = f"{reference:8.4f}" if reference else "       -"
    print(f"  {label:18s}  {f32:8.4f}      {extrapolated:8.4f}         {reference}"
          f"            {order:.2f}")
L_e_extrapolated = study[2]["Le"] + (study[2]["Le"] - study[1]["Le"]) / 3
```

```
                     h = 1/32   extrapolated   exact / correlation   observed order
  u_centre (outlet)     1.4967       1.4997          1.5000            1.96
  dp/dx (outlet)       -0.2397      -0.2402         -0.2400            1.88
  entrance length       2.4327       2.4308          2.3915            3.45
  extra drop            0.2231       0.2202               -            1.05
```



Entrance length and fully developed centre velocity on each grid, against h^2: second-order convergence gives a straight line.

- The fully developed velocity and pressure gradient converge with order close to 2 and extrapolate to the exact 1.5 and -0.24.
- The entrance length converges to $2.43\,H$, 1.6 % above the correlation of Durst et al.; its observed order is irregular because L_e is found by interpolating where a curve crosses the 99 % line.
- The extra pressure drop converges only at first order, towards about $0.22\,\rho U^2$: the pressure is singular at the inlet corners.

Result. On the main grid ($h = H/16$) the flow becomes fully developed after $L_e = 2.44\,H$ at $Re = 50$ and $4.67\,H$ at $Re = 100$ (Durst et al.: $2.39\,H$ and $4.54\,H$; grid-independent FDM value at $Re = 50$: $2.43\,H$). Downstream the profile is the Poiseuille parabola with $dp/dx = -12\mu U/H^2$ (FDM: -0.2381 and -0.1191 against -0.24 and -0.12, converging at second order). The flow rate is conserved to eight decimals. The entrance region costs an extra pressure drop of about $0.23\,\rho U^2$ ($Re = 50$) and $0.27\,\rho U^2$ ($Re = 100$) on top of the fully developed drop.

Key ideas

- With inlets and outlets, solve for the **primitive variables** u, v, p; the pressure is what keeps the velocity divergence-free.
- **Projection method:** predictor without pressure → pressure Poisson equation → correction; march to the steady state.
- The **staggered grid** couples neighbouring pressures, so no checkerboard can hide; a collocated grid with central differences cannot see one.
- **Boundary conditions:** velocity given (inlet, walls) → $\partial p/\partial n = 0$; pressure given (outlet) → $\partial u/\partial n = 0$.
- On a long domain, SOR needs help: a **block correction** with the Thomas algorithm removes the slow, column-wide errors.